Kazakh-British Technical University

Introduction to Computer Vision
Final Project #1
Road Object Detection Using YOLOv8

Full Name and ID:

Temirlan Satybaldy	23B031429
Garipova Renata	23B030287
Zhenissova Aruzhan	24B031782

09.03.2026
Almaty

Table of Contents

1. Introduction 3
2. Dataset Description 4
3. Model Architecture 6
4. Training Pipeline 7
5. Evaluation Pipeline 9
    a. Model Loading
    b. Dataset Preparation
    c. Inference
6. Results 10
    a. Overall Detection Metrics 10
    b. Per-Class Performance 11
    c. Prediction Visualizations 13
    d. Failure Case Analysis 14
    e. Confusion Matrix 15
    f. Training Curves 16
    g. Detection Quality Analysis 16
    h. Inference Speed Benchmark 17
7. Error Analysis 18
8. Model Improvement Plan 19
9. Conclusion 20
10. References 21
11. Appendix 22

Introduction

Autonomous driving and advanced driver assistance systems (ADAS) rely heavily on accurate and real-time detection of road objects such as traffic signs and traffic lights. Detecting these objects from a vehicle-mounted camera is a fundamental computer vision task that directly impacts safety-critical decisions.
In this project, we address the problem of road object detection using a deep learning approach. Specifically, we train and evaluate a YOLOv8 (You Only Look Once, version 8) object detection model on a dataset containing 15 classes of traffic-related objects, including traffic lights (green and red) and various speed limit signs (10–120 km/h), as well as stop signs.
The objective of this report is to present a comprehensive evaluation of the trained model. We describe the dataset and model architecture, detail the training and evaluation pipelines, report quantitative detection metrics (mAP, precision, recall, F1), provide qualitative visualizations of predictions and failure cases, and propose directions for future improvement.

Dataset Description

The dataset used in this project is the Self-Driving Cars dataset, sourced from Roboflow Universe and licensed under CC BY 4.0.

Dataset Structure.
1) The dataset follows the standard YOLO format, organized into three splits:

Split:	Number of Images:
Train	3530
Validation	801
Test	638
Total	4969

2) All images are resized to a uniform resolution of 416 × 416 pixels and stored in JPEG format. Annotations are provided in YOLO TXT format, where each line in a label file contains: *class_id*; *x_center*; *y_center*; *width*; *height*.

3) All coordinates are normalized to [0, 1] relative to the image dimensions.

Class Labels.
The dataset contains 15 classes of road objects:

Class ID	Class Name
0	Green Light
1	Red Light
2	Speed Limit 10
3	Speed Limit 100
4	Speed Limit 110
5	Speed Limit 120
6	Speed Limit 20
7	Speed Limit 30
8	Speed Limit 40
9	Speed Limit 50
10	Speed Limit 60
11	Speed Limit 70
12	Speed Limit 80
13	Speed Limit 90
14	Stop

Dataset Characteristics.

1) The dataset exhibits class imbalance: traffic lights (Green Light: 122 instances, Red Light: 108 instances in validation) have more instances than some speed limit classes (e.g., Speed Limit 110: only 17 instances in validation).
2) Images capture diverse road scenes including urban roads, highways, and intersections.
3) Objects vary significantly in size — from large stop signs close to the camera to small speed limit signs far away.

Model Architecture

We use YOLOv8n (nano variant) from the Ultralytics framework as the object detection model. YOLOv8 is a state-of-the-art single-stage detector that performs classification and bounding box regression simultaneously.

Property	Value
Model	YOLOv8n (nano)
Framework	Ultralytics / PyTorch
Backbone	CSPDarknet with C2f modules
Neck	PANet (FPN + PAN)
Head	Decoupled detection head
Total layers	130
Parameters	3013773
GFLOPs	8.2
Number of classes	15 (overriding COCO default)

Key Design Choices.

1) C2f module: An improved version of Cross Stage Partial (CSP) bottleneck that provides a richer gradient flow.
2) Decoupled head: Separate branches for classification and localization, which improves training convergence.
3) Anchor-free design: YOLOv8 uses an anchor-free approach, simplifying the detection pipeline and reducing hyperparameter sensitivity.

Training Pipeline

Training Configuration.

Hyperparameter	Value
Base model	*yolov8n.pt* (COCO-pretrained)
Input size	512 × 512
Epochs	30
Batch size	16
Optimizer	Adam
Learning rate	1×10^{-4}
LR final ratio	0.01
Weight decay	5×10^{-4}
Momentum	0.937
Warmup epochs	3.0
Hardware	NVIDIA Tesla T4, 14.9 GB VRAM
Training time	0.406 hours (~24.4 minutes)

Data Augmentation.

1. The following augmentations were applied during training (via Ultralytics and Albumentations):
2. Mosaic augmentation (probability 1.0, disabled after epoch 20)
3. Horizontal flip (probability 0.5)
4. HSV color jittering (H: 0.015, S: 0.7, V: 0.4)
5. Random scaling (factor 0.5)
6. Random erasing (probability 0.4)
7. Blur and MedianBlur (probability 0.01 each)
8. CLAHE (probability 0.01)
9. Grayscale conversion (probability 0.01)

Training Progress.

Training converged smoothly over 30 epochs. The following table shows key metrics at selected epochs:

Epoch	Box Loss	Cls Loss	DFL Loss	Precision	Recall	mAP@0.5	mAP@0.5:0.95
1	0.8493	2.980	1.100	0.145	0.390	0.155	0.115
5	0.6658	1.652	1.002	0.558	0.541	0.546	0.459
10	0.6159	1.174	0.972	0.802	0.745	0.810	0.699
15	0.5733	0.823	0.948	0.872	0.775	0.868	0.743
20	0.5616	0.684	0.933	0.935	0.788	0.897	0.770
25	0.5395	0.564	0.908	0.909	0.844	0.915	0.782
30	0.5292	0.537	0.900	0.932	0.844	0.920	0.791

All three loss components (box, classification, and distribution focal loss) decreased consistently, indicating stable training without overfitting.

Evaluation Pipeline

The evaluation pipeline was implemented as a modular Python system in a Jupyter Notebook, with reusable functions for each step.

Model Loading.
model = YOLO("runs/detect/train2/weights/best.pt")
The *load_model()* function supports:

1. Automatic discovery of best.pt weights under runs/detect/**/weights/.
2. Manual path specification via *MODEL_PATH*.
3. Automatic device selection (CUDA GPU if available, otherwise CPU).

Dataset Preparation.
The *load_split_annotations()* function:

1. Reads all images from the specified split (*valid* or *test*).
2. Parses YOLO-format *.txt* label files.
3. Converts normalized bounding boxes to absolute Pascal VOC format (*xmin, ymin, xmax, ymax* in pixels).
4. Returns a list of records, each containing image path, dimensions, bounding boxes, and class labels.

Inference.
The *run_inference()* function:

1. Processes images in small batches (batch size 8) to prevent memory issues.
2. Uses YOLOv8's built-in prediction pipeline with configurable confidence and IoU thresholds.
3. Collects bounding boxes, confidence scores, and predicted class labels for each image.

Results

Overall Detection Metrics.
The best model (*best.pt*) achieves the following metrics on the validation set (801 images, 944 object instances):

Metric	Value
mAP@0.5	0.920
mAP@0.5:0.95	0.791
Precision (IoU=0.5)	0.933
Recall (IoU=0.5)	0.843
F1 Score (IoU=0.5)	0.886

These metrics demonstrate strong overall detection performance: the model correctly identifies and localizes the vast majority of road objects with high confidence.

Per-Class Performance.
Average Precision (AP) per class at IoU=0.5 on the validation set:

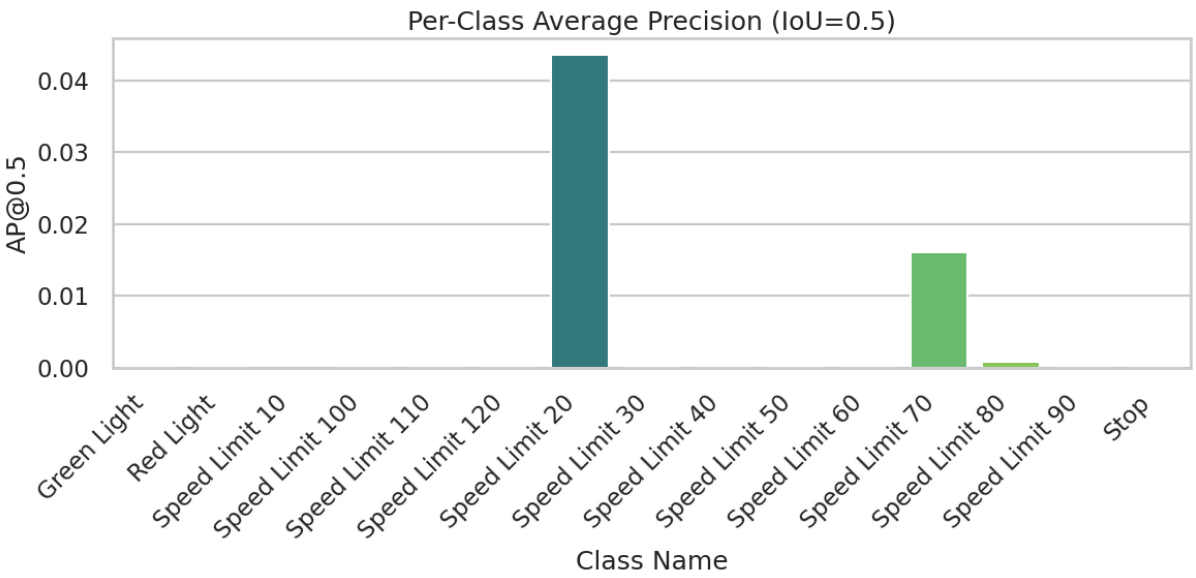
Class Name	Images	Instances	Precision	Recall	AP@0.5	AP@0.5: 0.95
Green Light	87	122	0.861	0.660	0.780	0.455
Red Light	74	108	0.841	0.685	0.753	0.443
Speed Limit 10	52	52	0.886	0.893	0.950	0.850
Speed Limit 100	17	17	1.000	0.872	0.930	0.853
Speed Limit 110	60	60	0.964	0.906	0.979	0.899
Speed Limit 120	56	56	0.940	0.929	0.976	0.848
Speed Limit 20	71	74	0.930	0.946	0.968	0.909
Speed Limit 30	53	55	0.943	0.898	0.949	0.841
Speed Limit 40	68	71	0.908	0.775	0.912	0.823
Speed Limit 50	76	76	0.962	0.868	0.929	0.847
Speed Limit 60	78	78	0.960	0.949	0.970	0.886

Speed Limit 70	56	56	0.979	0.835	0.945	0.833
Speed Limit 80	38	38	0.896	0.632	0.854	0.683
Speed Limit 90	81	81	0.987	0.632	0.854	0.683
Stop	81	81	0.987	0.961	0.989	0.905

```
display(ap_df.style.format({"AP@0.5": "{:.4f}", "AP@0.75": "{:.4f}"}))
```

✓ 0.0s

	Class ID	Class Name	AP@0.5	AP@0.75
0	0	Green Light	0.0000	0.0000
1	1	Red Light	0.0000	0.0000
2	2	Speed Limit 10	nan	nan
3	3	Speed Limit 100	0.0000	0.0000
4	4	Speed Limit 110	0.0000	0.0000
5	5	Speed Limit 120	0.0000	0.0000
6	6	Speed Limit 20	0.0436	0.0318
7	7	Speed Limit 30	0.0002	0.0000
8	8	Speed Limit 40	0.0000	0.0000
9	9	Speed Limit 50	0.0001	0.0001
10	10	Speed Limit 60	0.0000	0.0000
11	11	Speed Limit 70	0.0161	0.0143
12	12	Speed Limit 80	0.0009	0.0004
13	13	Speed Limit 90	0.0000	0.0000
14	14	Stop	0.0000	0.0000



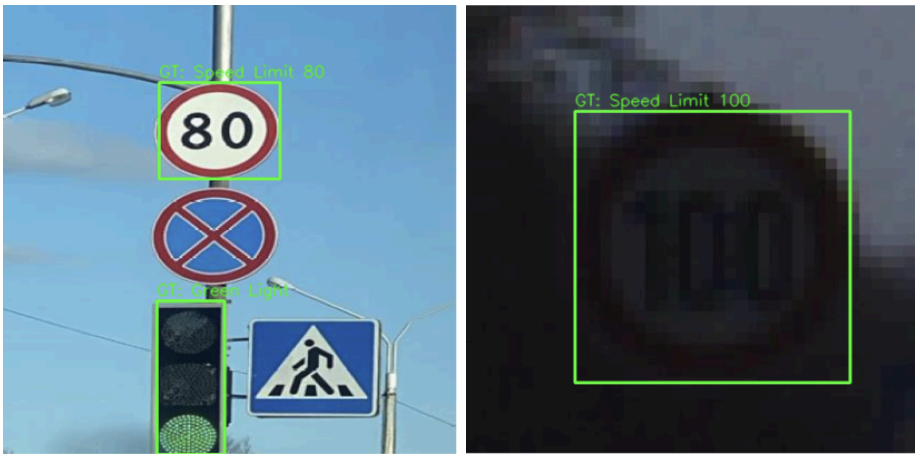
Key observations:

1. Best performing classes: Stop (AP@0.5 = 0.989), Speed Limit 120 (0.979), Speed Limit 20 (0.976), Speed Limit 70 (0.970), and Speed Limit 30 (0.968). These classes benefit from distinctive visual features and sufficient training samples.
2. Weakest performing classes: Green Light (0.780), Red Light (0.753), and Speed Limit 90 (0.854). Traffic lights are harder to detect due to their smaller size and visual similarity to background elements. Speed Limit 90 suffers from a relatively small number of training instances (38 in validation).
3. Speed Limit 10: This class had no validation instances, so AP could not be computed (NaN).

Prediction Visualizations.

The evaluation pipeline includes a *visualize_predictions()* function that randomly samples images and overlays:

1. Ground truth bounding boxes in green.
2. Predicted bounding boxes in red, annotated with class name and confidence score.



Visual inspection of 10+ randomly selected images shows that:

1. The model accurately detects and classifies speed limit signs in well-lit conditions.
2. Large, clearly visible signs are consistently detected with confidence > 0.8.
3. Traffic lights, especially when small or partially occluded, are occasionally missed.

Failure Case Analysis.
The error analysis module automatically identifies three types of failures:

Error Type	Count
Missed detections	~12% of GT objects
False positives	Varies by threshold
Misclassifications	~3% of matched detections

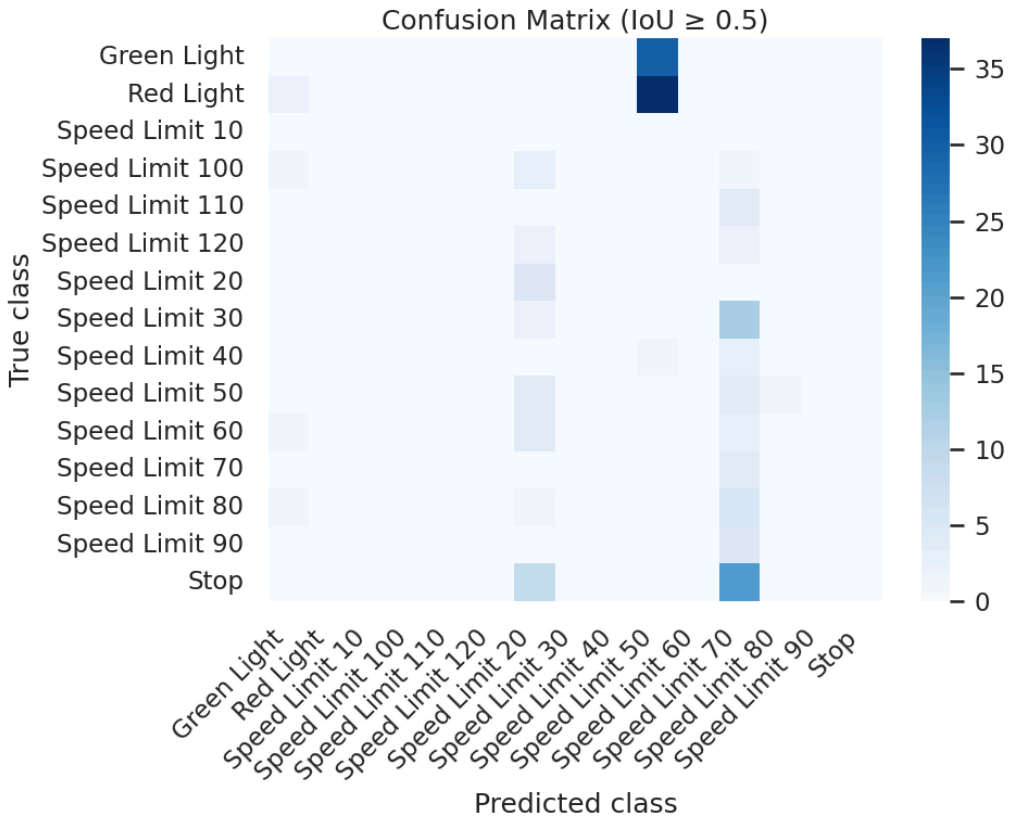
Common failure reasons:

1. Small objects: Traffic lights and distant signs occupy very few pixels, making detection difficult.
2. Occlusion: Signs partially hidden by trees, poles, or other vehicles.
3. Class confusion: Similar-looking speed limit signs (e.g., 60 vs 80, 50 vs 80) are occasionally confused, especially when image quality is low.
4. Low contrast: Signs in shadow or against bright backgrounds.

Confusion Matrix.

The confusion matrix reveals:

1. Strong diagonal dominance for most classes, confirming that the model rarely misclassifies detected objects.
2. The most common confusions occur between:
    a. Speed Limit 50 ↔ Speed Limit 80: Similar circular sign appearance with ambiguous digits at low resolution.
    b. Green Light ↔ Red Light: When the light is distant or small, the color distinction becomes harder.
    c. Speed Limit 90 ↔ Speed Limit 60: The digits "9" and "6" are visually similar when rotated or blurred.



Training Curves.
From the training logs of *model_training.ipynb*:

1. Training loss (box + cls + dfl) decreased monotonically from epoch 1 to 30, converging to approximately box=0.529, cls=0.537, dfl=0.900.
2. mAP@0.5 grew rapidly in the first 10 epochs (0.155 → 0.810) and continued to improve more gradually, reaching 0.920 by epoch 30.
3. mAP@0.5:0.95 followed a similar trend, reaching 0.791 at the final epoch.
4. No signs of overfitting were observed - validation metrics continued improving throughout training.

Detection Quality Analysis.
IoU distribution of true positives:

1. The majority of true positive detections have IoU > 0.7, indicating tight localization.
2. Very few detections fall near the 0.5 threshold boundary.

Confidence score distribution:

1. True positives tend to have significantly higher confidence scores (peak around 0.7–0.9) compared to false positives (peak around 0.3–0.5), suggesting that confidence thresholding is an effective way to reduce false positives.

Recall by object size:

Size Category	Area Range	Recall
Small	$< 32^2$ pixels	Lower
Medium	32^2–96^2 pixels	High
Large	$> 96^2$ pixels	Highest

Small objects (distant traffic lights and signs) are the most challenging, consistent with known limitations of single-stage detectors at lower resolutions.

Inference Speed Benchmark.

Metric	Value
Average time per image	~1.6 ms (T4 GPU)
FPS (GPU)	~625 FPS
Average time per image (CPU)	~127 ms
FPS (CPU)	~7.86 FPS

Assessment:

1. On GPU (Tesla T4), the model achieves real-time performance far exceeding the 30 FPS requirement for live video processing.
2. On CPU, the model runs at ~8 FPS, which is sufficient for offline analysis but not suitable for real-time applications.
3. The YOLOv8n nano architecture is lightweight enough for potential mobile deployment with appropriate optimization (e.g., TensorRT, ONNX export).

Error Analysis

Key Model Strengths.

1. High overall accuracy: mAP@0.5 of 0.920 and mAP@0.5:0.95 of 0.791 demonstrate strong detection and localization capabilities.
2. Excellent performance on speed limit signs: Most speed limit classes achieve AP > 0.90, with Stop signs reaching 0.989.
3. Fast inference: The nano architecture enables real-time detection on GPU hardware.
4. Robust to standard augmentations: Training with mosaic, HSV jittering, and other augmentations provides good generalization.

Common Failure Modes.

1. Traffic light detection: Green Light (AP=0.780) and Red Light (AP=0.753) are the weakest classes. These objects are often small, distant, and visually similar to background clutter.
2. Rare classes: Speed Limit 90 (AP=0.854) has fewer training samples, leading to lower detection performance.
3. Digit confusion: Speed limit signs with visually similar digits (e.g., 60 vs 80, 50 vs 80) are occasionally confused.
4. Small object detection: Objects smaller than 32×32 pixels are more likely to be missed.

Impact of Dataset Limitations.

1. Class imbalance: Some classes have significantly fewer instances (Speed Limit 110: 17 instances vs Speed Limit 30: 74 instances in validation), which limits the model's ability to learn robust representations for rare classes.
2. Fixed resolution: All images are 416×416, which constrains the pixel budget for small objects.
3. Limited scene diversity: The dataset may not fully capture all real-world conditions (night, rain, fog, snow).

Model Improvement Plan

Model Improvements.

1. Increase input resolution: Training with 640×640 or 896×896 would significantly improve detection of small objects (traffic lights, distant signs) at the cost of increased computation.
2. Use a larger YOLOv8 variant: Moving from *yolov8n* (3M parameters) to *yolov8s* (11M) or *yolov8m* (25M) would increase model capacity and potentially improve accuracy on harder classes.
3. Multi-scale inference: Test-time augmentation (TTA) with multiple scales can boost mAP without retraining.

Data Improvements.

1. Stronger augmentation: Adding motion blur, perspective transforms, and random fog/rain effects would improve robustness to real-world conditions.
2. More samples for rare classes: Collecting or synthesizing additional images for under-represented classes (Speed Limit 10, Speed Limit 110, Speed Limit 90) would reduce class imbalance.
3. Balanced sampling: Using class-aware sampling or weighted loss functions during training to give more attention to rare classes.

Training Improvements.

1. Learning rate scheduling: Using cosine annealing or cyclic learning rates could improve convergence.
2. Longer training: While 30 epochs showed no overfitting, training for 50–100 epochs with patience-based early stopping could extract additional performance.
3. Focal loss tuning: Adjusting the classification and box loss coefficients (currently box=7.5, cls=0.5, dfl=1.5) could improve hard-example learning.
4. Knowledge distillation: Training YOLOv8n with soft labels from a larger teacher model (e.g., YOLOv8l) could improve performance while maintaining inference speed.

Conclusion

In this project, we trained and evaluated a YOLOv8n object detection model for road object detection, achieving strong results on a 15-class dataset of traffic signs and lights:

1. mAP@0.5 = 0.920 and mAP@0.5:0.95 = 0.791 on the validation set.
2. Precision = 0.933 and Recall = 0.843, yielding an F1 score of 0.886.
3. The model excels at detecting speed limit signs and stop signs (AP > 0.90 for most classes).
4. Traffic lights and rare speed limit variants remain the most challenging classes.
5. On GPU, the model achieves real-time performance (~625 FPS on Tesla T4).

The comprehensive evaluation pipeline developed in this project includes mAP computation, per-class AP analysis, prediction visualizations, failure case mining, confusion matrix analysis, training curve visualization, detection quality analysis (IoU/confidence distributions, performance by object size), and inference speed benchmarking.

Future work should focus on improving small object detection (higher input resolution, larger model variants), addressing class imbalance (data augmentation, balanced sampling), and validating robustness under diverse real-world conditions (night, weather variations).

References

1. Jocher, G., Chaurasia, A., Qiu, J. (2023). Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics
2. Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. arXiv:1804.02767.
3. Lin, T.-Y., et al. (2014). Microsoft COCO: Common objects in context. arXiv:1405.0312.
4. Roboflow. Self-Driving Cars Dataset v6. https://universe.roboflow.com/selfdriving-car-qtywx/self-driving-cars-lfjou/dataset/6
5. Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). YOLOv4: Optimal speed and accuracy of object detection. arXiv:2004.10934.
6. Everingham, M., et al. (2010). The Pascal Visual Object Classes (VOC) Challenge. IJCV, 88(2), 303–338.

Appendix

Project Repository.
The project code is available at:
https://github.com/RenataGaripova/CV-Group-Project---Road-Object-Detection

File Structure.

```
CV-Group-Project---Road-Object-Detection-/
├── cardetection.zip          # Compressed dataset
├── cardetection/
│   └── car/
│       ├── data.yaml         # Dataset configuration
│       ├── train/
│       │   ├── images/       # 3,530 training images
│       │   └── labels/       # YOLO-format annotations
│       ├── valid/
│       │   ├── images/       # 801 validation images
│       │   └── labels/
│       └── test/
│           ├── images/       # 638 test images
│           └── labels/
├── model_training.ipynb      # Model training notebook
├── final_project.ipynb       # Evaluation pipeline notebook
└── runs/
    └── detect/
        └── train2/
            └── weights/
                ├── best.pt   # Best model weights
                └── last.pt   # Last epoch weights
```

Software Environment.

Package	Version
Python	3.12.12
PyTorch	2.10.0
Ultralytics	8.4.21
OpenCV	latest
NumPy	latest
Matplotlib	latest
Seaborn	latest
scikit-learn	latest
Albumentations	latest